

JavaScript Fundamentals

JavaScript: A language that implements the interactivity, behavior, and logic of a web page.

In JS, variables can be declared with one of three keywords:

- `var` is outdated since the ES6 revision of JavaScript in 2015.
- `const` should be used when the value is a constant that you don't expect to change.
- `let` is used when you do expect changes.

Array: The list data type.

Sample JavaScript syntax:

```
while (lives > 0) {  
    console.log("waka waka")  
    x -= 1  
}
```

```
for (statement 1; statement 2; statement 3) {  
    /* code block */  
}
```

- `statement 1` is run a single time before the first iteration and is used to initialize a variable.
- `statement 2` provides a condition for continued iteration.
- `statement 3` increments the variable.

```
if ( lives == 0 ) {  
    alert( messages[1] )  
} else {  
    console.log( messages[3] )  
}
```

```
function add( x, y ) {  
    return x + y  
}
```

Common JS Methods

`alert()` displays a pop-up message in the browser.

`console.log()` logs information in the browser's console.

`prompt()` displays a pop-up message, allows the user to enter a response, then returns the user's response as a value.

To call the `add` function with inputs of 2 and 3, simply call `add(2, 3)`.

Scope: Defines the areas in a program where a variable can be seen and accessed.

Block scope: Refers to the scope of a variable declared inside a code block that can only be referenced from within the same code block.

Global scope: Refers to the scope of a variable declared outside of all code blocks that can be referenced from anywhere in the entire program.

Hoisting: JavaScript pulls all function definitions to the top of a script, allowing them to be called before they're declared in the program.

Synchronicity: The order in which each statement in a program is executed.

In synchronous programming languages, each statement is executed one at a time from top to bottom.

Asynchronous functions can execute out of order. Other statements won't wait for them to finish.

JavaScript Objects

Object: Code that stores related data and functionality.

Property: A characteristic of an object, i.e., data.

Method: An action an object can perform, i.e., functionality.

Object-oriented programming (OOP): A programming paradigm in which a program's data and functionality is organized into various objects.

Object literals: Individual objects in which each property and method is explicitly defined.

Constructor: A function that serves as a template for constructing objects.

Prototypes: Built-in JavaScript objects that provide a set of generic properties and methods that every object of a particular type inherits.

Creating an Object Literal

```
const course1 = {  
  name: "MATH 101",  
  get_name() {  
    return this.name  
  }  
}
```

Creating a Constructor

```
function Enemy(type){  
  this.type = type;  
  this.move = function(){  
    /* code */  
  };  
}
```

The Document Object Model

Document Object Model (DOM): The interconnected objects in an HTML document, represented by a tree-like structure. The DOM provides a way for external scripts and programs to manipulate the content on a web page.

Nodes: Objects in the DOM.

Nearly all the relevant content in the DOM falls under one of these nodes: **Document**, **Element**, **Attribute**, and **Text**.

Document is the entry point into the DOM because its properties and methods provide for access and manipulation of any node in the document.

Manipulating the First h2 Object Through the DOM

```
const $h2 = document.  
querySelector("h2")  
$h2.textContent = "Hi!"
```

Events and Forms

Events: Occurrences within a program's environment, like a user clicking a button or an error.

The built-in **addEventListener** method "listens" for an event and runs a function when that event occurs. It requires two arguments:

1. The type of event (e.g., "click," "copy," "drag")
2. A reference to a function, also called a handler function

Functions used in event listeners use the **this** keyword to refer to the object with the attached event listener.

Form element: A section of the document that provides controls for collecting user input, created with **<form>**.

Input element: Creates interactive UI components like text boxes, checkboxes, and file upload buttons that gather user input.

Label element: An optional element that provides a label for any input element it's paired with.

Creating a Button in HTML

```
<button>text</button>
```

Creating a Form in HTML

```
<form class="bottom">  
  <input type="text"  
    id="myInput"  
    placeholder="task">  
  <button id="addBtn">  
    Add</button>  
</form>
```

Interaction Design

UX design: Every user-facing aspect of a product, i.e., its entire user experience.

Interaction design: The interactive aspects of a product's user experience, or how users and the system communicate.

Jakob Nielsen's ten usability heuristics:

1. Visibility of system status: A system's state should be conveyed to its users.
2. Match between the system and the real world: A system should use language, features, and concepts that users may already be familiar with from the real world.
3. User control and freedom: A system should give users opportunities to revert to any previous state.
4. Consistency and standards: Follow web design standards so users can spend less time learning how to use your product and more time performing their desired tasks.
5. Error prevention: Systems should implement preventative measures to help users avoid errors.
6. Recognition rather than recall: Systems should rely on recognition whenever possible.
7. Flexibility and efficiency of use: A system should provide features and shortcuts that cater to users of different experience levels.
8. Aesthetic and minimalist design: A system's design should balance aesthetic appeal with informational clarity.
9. Help users recognize, diagnose, and recover from errors: In cases of error, users should be told what went wrong and how it can be fixed.
10. Help and documentation: A system should provide resources that proactively and retroactively help users learn the product.

Five Aspects of the User Experience

- Words
- Visuals
- Physical space
- Time
- Behavior

Front-End Libraries and Frameworks

Import and export: Keywords to pass functionality between files.

Library: A collection of related code that a developer can pull from to perform specific tasks or implement specific functionality.

Framework: A whole ecosystem of tools and standards that serve as scaffolding that gives an application structure.

jQuery: The most popular JavaScript library.

React: The leading JavaScript front-end framework.